

GuideFetch: A Task Coordination Framework for Concurrent Navigation and Object Retrieval in Assistive Robot Dogs

Qian Yin¹, Ruiping Liu^{1,*}, Kunyu Peng^{1,*}, Jianxiang Man¹, Isik Baran Sandan¹, Junwei Zheng¹, Yufan Chen¹, Di Wen¹, Kailun Yang², and Rainer Stiefelhagen¹

¹ Karlsruhe Institute of Technology, Germany

² Hunan University, China

Abstract. Consider a robot guide dog escorting a blind user to an available seat while a second assistive robot dog concurrently retrieves a cup of coffee and delivers it to the same seat. This setting motivates concurrent execution because navigation and object retrieval can overlap. A syntactically valid Large Language Model (LLM) plan may still violate embodiment constraints, and successful-looking controller motion does not by itself establish task completion. We introduce GUIDEFETCH, a coordination framework for concurrent navigation and object retrieval by a heterogeneous guider and fetcher team. An LLM instantiates a schedule-conditioned four-action schema from a natural-language instruction. Before execution, robot, skill, and target aliases are normalized, and proposed actions are validated against registered targets, robot capabilities, and the selected schedule. Robot and object states then govern sequential and parallel execution. In a matched 2×2 study across 90 combinations of scene and seed (360 executions), all 180 online LLM responses validate without fallback or replay and match the corresponding scripted plans. For each planner source, sequential and parallel execution achieve 72/90 and 71/90 operational successes, respectively. Among the 56 cases completed by both schedules, parallel execution reduces mean makespan by 41.3%. Within this controlled setting, role specialization and action overlap shorten completed missions, while state checks distinguish plan validity from verified mission completion. Source code will be available.

Keywords: assistive robotics · heterogeneous multi-robot systems · task coordination · large language models · legged manipulation · physical verification

1 Introduction

Robot guide dogs are increasingly expected to support blind and low-vision users beyond simple point-to-point navigation, including tasks that require retrieving and delivering everyday objects. Consider one such scenario: when the user wants

* **Corresponding authors:** ruiping.liu@kit.edu; kunyu.peng@kit.edu

to rest, one guide dog leads them to an available seat while a second assistive robot dog is dispatched to a coffee machine, retrieves a cup, and delivers it to the same seat. Realizing this behavior requires a robot team to satisfy navigation and object-retrieval objectives within a single mission, even though the two capabilities belong to different embodiments. Assistance dogs already exhibit this kind of functional specialization in practice, with guide, hearing, and service dogs each trained for a distinct task [1]. In our setting, a *guider* completes the navigation objective while a *fetcher* acquires and delivers the object. Once these responsibilities are distributed across heterogeneous robots, however, coordination requires more than executing two independent skill sequences: each action must be assigned to a capable embodiment, temporal dependencies must be respected, conflicting commands to the same robot must be prevented, and mission progress must be verified from measured terminal conditions rather than assumed from plan validity or motion smoothness alone.

The two capabilities underlying this scenario have largely developed separately. Quadruped guide robots focus on wayfinding, interaction, and user safety [13], whereas assistive mobile manipulators retrieve and deliver objects [14]. Language-grounded systems connect one request to predefined skill affordances [10], executable programs [16, 27], and execution feedback [9]. Multi-robot extensions use large language models (LLMs) for decomposition, dialogue, and capability-aware allocation [4, 12, 18, 19]. Yet one question remains for guide-and-fetch assistance. Can heterogeneous robots exploit concurrency while ensuring that symbolic progress corresponds to verified task completion?

The generated plan alone cannot answer this question. A valid-looking plan may assign grasping to a robot without a manipulator. A completed trajectory may leave the object unsupported, while an early failure may make a parallel run appear fast. The guider must reach its destination, and carrying must remain blocked until the fetcher reaches pickup, establishes contact, and lifts the object. We therefore treat plan validity, execution progress, and mission completion as distinct states.

GUIDEFETCH separates task specification from robot execution and measured completion. An LLM instantiates a schedule-conditioned schema over registered actions. A deterministic validator resolves aliases and rejects assignments that violate robot capabilities or the selected schedule. The accepted records are compiled into nominal dependency levels, and branch-local state machines execute under one simulation clock. Navigation arrival, bilateral gripper contact, object lift and retention, and assisted delivery are recorded as terminal events that govern branch progress and mission success. The LLM supplies structured action records, while deterministic checks and embodied state govern execution and completion.

We instantiate GUIDEFETCH in InternUtopia, built on GRUtopia [29], with a Unitree Go2 guider and a Unitree B2 fetcher equipped with a Z1 arm. The central experiment tests whether role specialization shortens the mission without changing its terminal objectives. Sequential execution assigns the unloaded destination visit, pickup, and delivery to the fetcher. Parallel execution assigns

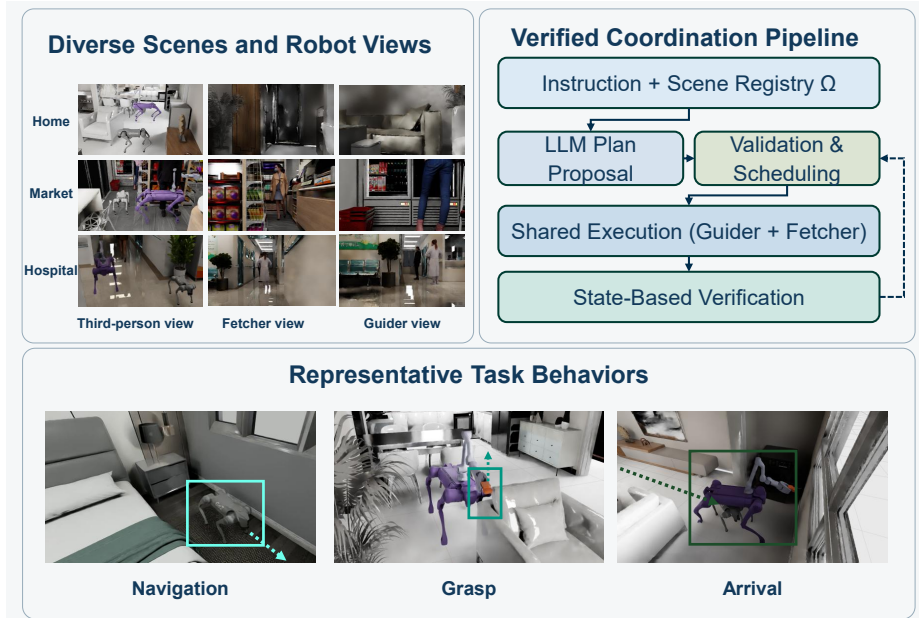


Fig. 1: Overview of GUIDEFETCH. Left: three evaluation scenes and robot views. Right: language-conditioned planning, deterministic validation and scheduling, shared execution of the guider and fetcher, and state-based verification. Bottom: representative navigation, grasp, and arrival behaviors.

the unloaded visit to the guider while the fetcher begins retrieval. Crossing both schedules with scripted and newly generated online LLM plans over 90 matched combinations of scene and seed yields 360 executions across Home, Supermarket, and Hospital.

All 180 online LLM responses pass validation without fallback or replay and match the corresponding scripted plans within the tested templates. For each planner source, sequential and parallel execution achieve 72/90 and 71/90 operational successes. In the 56 cases completed by both schedules, parallel execution is always faster and reduces mean makespan from 164.78 to 96.79 s, or 41.3%. All failures occur during pre-transport manipulation, including grasp, lift, and retention, while every parallel guider reaches its goal. The results attribute the timing benefit to redistributed and overlapping embodied work and identify manipulation as the remaining bottleneck.

2 Related Work

Assistive navigation and object retrieval. Robot guidance and robotic object assistance address complementary needs, but they have often been developed as separate systems. ARP [13] formalizes wayfinding, interaction, and safety

for a quadruped guide robot. Dusty [14] demonstrates object retrieval with an assistive mobile manipulator. More recent navigation work studies spatial language maps [8], language-guided semantic graphs [15], voice interaction with quadruped guide robots [21], and language and vision cooperation for assistive mobile robots [33]. On the manipulation side, language has been used to reason about object orientation [22], grasp affordances [5], and fine-grained object affordances [30]. We combine registered destinations and objects with existing controllers to study how a guider and fetcher complete one request at a shared destination.

From instructions to executable robot behavior. Language-based integration requires a user request to become actions that each robot can execute. SayCan [10] ranks predefined skills using language and learned affordances. Code as Policies [16] and ProgPrompt [27] generate programs over controlled robot interfaces, while SayPlan [25] grounds longer plans in a three-dimensional (3D) scene representation. Inner Monologue [9] incorporates observations during planning, and recent systems also study verification or repair during execution [7, 17, 28]. A well-formed output, however, does not prove that the assignee has the required capability or that an object was lifted and delivered. In GUIDEFETCH, an LLM instantiates a schedule-conditioned four-action schema, deterministic checks validate its assignments, the compiler constructs its dependency structure, and embodied state determines progress and completion.

Task allocation in heterogeneous robot teams. An executable skill set still leaves the team-level decision of which robot should act and when. Kannan et al. [12] form coalitions from robot capabilities, Madi et al. [19] use inter-robot dialogue and collision feedback, and Liu et al. [18] integrate planning and execution for heterogeneous teams. Embodiment-aware work represents robot capabilities explicitly [4], while centralized and decentralized LLM architectures are compared in [6]. Related work combines language reasoning with formation control [32], cooperative transport [31], and search over joint plans [34]. Our experiment isolates a narrower question. When guidance and retrieval belong to different embodiments, can heterogeneous role assignment and action overlap shorten the mission while preserving its terminal objectives? We compare sequential and parallel schedules under the same controllers and state-based terminal checks.

Embodied simulation and controlled evaluation. The scheduling comparison requires matched initial conditions and physical perturbations while preserving the terminal objectives. Habitat 3.0 [20] supports collaborative tasks involving humans and robots, and Chang et al. [3] evaluate planning and reasoning in embodied multi-agent tasks. GRUtopia [29] provides interactive scenes and configurable tasks for different robots. RoboFactory [23] introduces compositional constraints, while Kang et al. [11] study embodied multi-agent cooperation. Our study pairs both schedules by scene and seed while fixing the mission, controllers, and perturbations. The paired makespan comparison therefore reflects the implemented role assignment and action overlap among missions completed by both schedules.

3 Problem Formulation

We formulate the guide-and-fetch mission as a capability-constrained task graph and compare two schedules that preserve its terminal objectives while changing robot assignment and action overlap. Let $\mathcal{R} = \{r_G, r_F\}$ denote the guider (G) and fetcher (F), where each robot r_i provides a set $\mathcal{C}_i$ of available skills. The mission vocabulary is $\mathcal{A} = \{\text{navigate}, \text{move}, \text{grasp}, \text{carry}\}$, and the scene registry Ω contains the pickup station g_p , final station g_f , and task object o . Given an instruction x and a selected schedule, the planner instantiates four action records

$$V = \{v_j\}_{j=1}^4, \quad v_j = (k_j, r_j, a_j, z_j, p_j), \quad (1)$$

where a vertex records the nominal dependency-level index k_j , assignee r_j , skill $a_j \in \mathcal{A}$, target z_j , and additional parameters p_j . The validator checks the records against Ω , the capability sets, and the selected schedule. The compiler then constructs $\mathcal{G} = (V, E)$, where an edge $(v_j, v_\ell) \in E$ requires v_j to succeed before v_ℓ can start. The graph is admissible only when every robot and target is registered, each skill belongs to the assignee’s capability set, and the required action structure is preserved. During execution, every action has one runtime state

$$q_j \in \{\text{pending}, \text{running}, \text{succeeded}, \text{failed}\}. \quad (2)$$

An action can run only after its predecessors have succeeded and its robot is idle. Success releases the next action on the same branch. A failed prerequisite prevents its downstream actions from running and ends the evaluated episode. The LLM therefore supplies structured action records, while the compiler, capability checks, and runtime state determine execution.

The sequential and parallel schedules encode the same two objectives, an unloaded visit to g_f and delivery of o from g_p to g_f . Sequential execution assigns the complete mission to the fetcher

$$P_{\text{seq}} := [\text{F:move}(g_f), \text{F:move}(g_p), \text{F:grasp}(o), \text{F:carry}(o, g_f)]. \quad (3)$$

The fetcher first visits the final station without the object, returns to the pickup station, and then retrieves and delivers the object. Parallel execution assigns the unloaded visit to the guider and starts it together with the fetcher branch

$$P_{\text{par}} := \{\text{G:navigate}(g_f) \parallel [\text{F:move}(g_p), \text{F:grasp}(o), \text{F:carry}(o, g_f)]\}. \quad (4)$$

The branches start together, but actions inside the fetcher branch remain ordered. Grasping requires arrival at g_p , and carrying requires successful grasp and lift. Fetcher manipulation does not wait for guider arrival. The schedules thus represent the same guide-and-fetch outcome through different role assignments and temporal structures.

The scheduling hypothesis is that assigning the unloaded visit to the guider allows it to overlap with object retrieval and can shorten a successfully completed mission. Let T_F^{visit} and T_F^{return} denote the fetcher’s unloaded trip to g_f and return to g_p . Let T_F^{move} denote its direct move from the initial position to g_p . The

remaining fetcher durations are grasp, lift, and pre-transport retention, T_F^{grasp} , and loaded transport, T_F^{carry} . With guider navigation time T_G^{nav} , the schedule structures give

$$T_{\text{seq}} = T_F^{\text{visit}} + T_F^{\text{return}} + T_F^{\text{grasp}} + T_F^{\text{carry}}, \quad (5)$$

$$T_{\text{par}} = \max(T_G^{\text{nav}}, T_F^{\text{move}} + T_F^{\text{grasp}} + T_F^{\text{carry}}). \quad (6)$$

The durations are measured during execution and need not match across robots or scenes. The comparison therefore captures the implemented combination of heterogeneous role assignment and action overlap in the same mission.

Mission success requires state verification of every terminal objective, not only a valid plan or completed controller command. Let b_{visit} , b_{pickup} , b_{grasp} , b_{lift} , b_{retain} , and b_{delivery} be Boolean variables computed from controller or simulator state. Operational success is

$$y_{\text{op}} = b_{\text{visit}} \wedge b_{\text{pickup}} \wedge b_{\text{grasp}} \wedge b_{\text{lift}} \wedge b_{\text{retain}} \wedge b_{\text{delivery}}. \quad (7)$$

The visit and pickup variables record arrival at g_f and g_p . The remaining variables record grasp, lift, pre-transport retention, and assisted delivery under the state checks defined in Sec. 4. An action completes at the first simulator timestamp when its terminal predicate becomes true. Parallel makespan is the later of guider arrival and fetcher delivery, while sequential makespan ends at delivery after all preceding actions have completed. Route length remains a trajectory measurement and is not an additional success condition after goal arrival. A timeout or failed manipulation gate produces a failure record, never an artificially short completion time. Speed is therefore evaluated only after both the destination visit and assisted object delivery have been verified.

4 System Architecture

GUIDEFETCH turns one user instruction into a verified guide-and-fetch mission by validating schedule-conditioned action records, compiling their dependencies, executing the robot branches under a shared clock, and recording completion from robot and object states. Figure 1 summarizes this path from language input to embodied outcome.

4.1 Planning and Validation

Planning ends with a validated four-action schema, not with the raw LLM response. For each condition, the prompt lists the available robots, four permitted skills, registered targets, selected schedule, and required JavaScript Object Notation (JSON) format. Each proposed action contains a step, robot, skill, target, and parameter record. The JSON **step** field supplies the nominal dependency-level index k_j . The parser extracts one JSON object, normalizes robot, skill, and target aliases, and resolves the targets against the scene registry. The validator then checks that all four expected actions are present, each assignee provides the

requested skill, and the carry action retains the registered object. A parallel plan must contain guider navigation and fetcher motion at step 1, followed by fetcher grasp and carry. A sequential plan must contain four ordered fetcher actions. Extra or missing actions, incorrect targets, or incompatible robot assignments cause rejection before simulation. Rejected responses and validation messages are stored, and any new response undergoes the same checks. Scripted plans pass through the same parser and validator as online LLM outputs.

The compiler uses the validated step indices to construct nominal dependency levels and assigns each action a stable identifier stored in the run record. Parallel plans contain three levels with two, one, and one actions, while sequential plans contain four single-action levels. These levels describe the dependency structure and do not impose global synchronization barriers. The two initial parallel actions start together. Fetcher grasp and carry depend only on the preceding fetcher events, so guider arrival does not delay manipulation.

4.2 Schedule Execution

The executor advances all active robots under one simulation clock while each branch progresses from its own state events. Sequential execution instantiates only the fetcher, which visits the final station, returns to pickup, grasps and lifts the object, and then delivers it. Parallel execution instantiates both robots in the same InternUtopia task. Guider navigation and fetcher motion to pickup begin at the same simulation step. At every subsequent step, the executor updates the active controllers and evaluates their terminal predicates. A satisfied predicate is timestamped immediately and releases the next action on that branch. The guider may reach the final station and hold position while retrieval continues, and the fetcher may begin manipulation while the guider is still moving. No fixed waiting interval is inserted. The parallel mission completes only after both guider arrival and object delivery.

Failure propagation follows the same dependencies, so progress on one branch cannot conceal an unmet objective on the other. A pickup timeout prevents grasp and carry from starting. Missing bilateral contact or retained lift prevents assisted transport. A missing guider arrival leaves the visit objective unsatisfied even if the fetcher reaches the final station. The executor records the branch progress and first failed gate before ending the episode. Both schedules use the same fetcher controller, scene configuration, sampled perturbation, and completion thresholds.

4.3 Robot Skills

In the parallel schedule, the guider performs the unloaded destination visit, with success defined by goal attainment under the implemented registered-route controller. It uses a recurrent policy trained with Proximal Policy Optimization (PPO) [26] from 180 planar light detection and ranging (lidar) measurements and an 11-element state vector that encodes goal geometry, base velocity, body

orientation, and the previous action. During the shared mission, the policy contributes local planar commands and the visible gait, while a waypoint progress shield advances the root through kinematic transport along the registered route. Collision response for the transported root is disabled. The navigation condition is satisfied when the guider reaches within 0.20 m of the final waypoint, so this event records destination arrival and does not establish dynamic collision-aware locomotion.

The fetcher provides object acquisition and assisted delivery, with contact and lift verified before assisted transport begins. It combines a locomotion policy associated with Autonomous Large-Object Rearrangement with a Legged Manipulator (ALORE) [2] with joint-space arm staging and world-frame inverse kinematics. The ALORE-derived controller supplies leg commands and visible gait, while registered-route transport advances the base to pickup and delivery. The base stops before the arm aligns the gripper with the registered object handle. Grasp success requires contact on both sides of the object. Lift success requires a peak object rise of at least 30 mm. Retention is evaluated over a 1.0 s hold at 200 Hz and requires at least 20 mm retained rise, bilateral contact in at least 80% of hold frames and in the final frame, at least 5 mm separation between the target and support collision bounds, and at most 17° object tilt. The distance from the registered grasp reference to the end-effector frame origin must also remain within 0.25 m. This frame-origin bound is not a fingertip-clearance measure. Object pose synchronization with the end effector starts only after these pre-transport checks succeed. The initial contact, lift, and retention are measured from simulated state, while subsequent transport is assisted along a registered delivery route. Both schedules use the same manipulation sequence, thresholds, and assisted-delivery controller.

4.4 Completion Checks and Run Records

Completion checks provide one evidence layer for both schedules and connect the validated plan to the final mission outcome. The unloaded visit is satisfied when its assigned robot reaches the final station without the object. Pickup arrival is recorded when the fetcher completes its registered route to g_p . Grasp requires bilateral contact, lift requires the measured object rise, and retention requires the hold gate defined above. Delivery requires the fetcher and synchronized object state to reach the final station after that gate has passed. These measurements determine the variables in Eq. (7). A failed check ends the episode and prevents dependent actions from running. Route length and waypoint progress are stored for analysis but do not add a second success condition after goal arrival. Each run record stores the scene, seed, planner source, schedule, normalized plan, validation result, controller configuration, action transitions, terminal conditions, timings, and object lift measurements. An online LLM record also retains the raw response, model identifier, request latency, token counts, and rejected attempts. Common scene, seed, planner, and schedule identifiers link these fields to one episode, allowing a failure to be traced to planning, controller execution, or a state-based completion check.

5 Experiments

The experiments test the two claims at the center of the guide-and-fetch mission. The planner study asks whether different instruction phrasings produce the required executable action schema. The scheduling study asks whether heterogeneous role assignment and action overlap shorten missions that satisfy the same state-based completion conditions.

5.1 Implementation Details

The implementation fixes scene geometry and routes before any planner or schedule comparison. We use the official Home, Supermarket, and Hospital GRScenes environments. Figure 2 shows the registered robot starts, pickup location, shared goal, and role-specific routes in each scene. The object pose at pickup and all route waypoints are also registered. The registered paths satisfy the scene-specific clearance requirements and are shared by both planner sources. The longer Hospital delivery route is retained as part of the original scene layout.

All experimental conditions use the same simulator and controller settings. The simulator advances at 200 Hz, and both robots in the parallel condition share one physics clock. Background scene physics and collision response for route-based base transport are disabled, while robot articulation, target-object dynamics, and gripper contact remain active. The manipulation controller and its success thresholds are fixed across conditions. The object follows the end effector during transport only after the grasp, lift, and hold conditions have passed. Because collision response is inactive for the registered-route transport, collision statistics are not reported.

5.2 Experimental Design

The evaluation uses a matched 2×2 design that crosses planner source with execution schedule. The planner is either scripted or queried online, and the schedule is either sequential or parallel as defined in Eqs. (3) and (4). The online planner uses a locally served four-billion-parameter Qwen3.5 model [24]. All 180 plan artifacts record the deployment identifier `frob/qwen3.5-instruct:4b` and the same local chat endpoint. All four conditions use the same normalization, validation, controller checkpoints, state checks, and timeout. Every Qwen3.5 run makes a new request without fallback or recorded replay. Four semantically equivalent instruction templates per schedule are rotated across seeds to vary the wording while preserving the requested subgoals. We use 30 seeds in each of the three scenes. Each seed produces all four conditions, giving 90 matched sets and 360 separate simulator executions.

Every scene and seed is matched across the four conditions, so a comparison is not confounded by a different initialization. Seeded perturbations vary robot starts by no more than 10 mm, object translation by 1.5 mm, object yaw by 0.5° , and object mass between 5.00 and 5.25 g. Contact friction is also varied within a narrow calibrated range. The exact perturbation record is generated

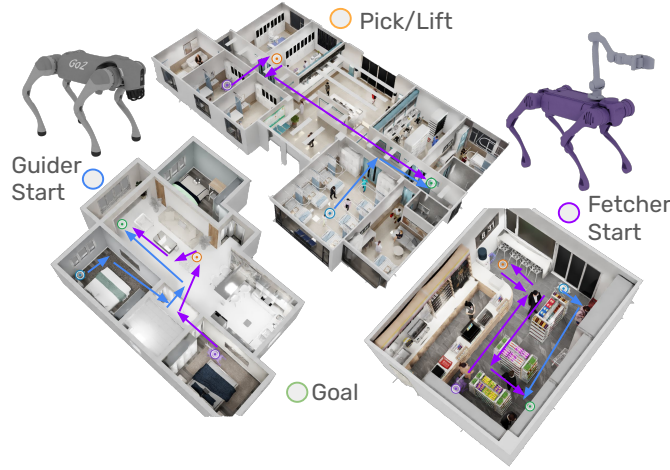


Fig. 2: Registered layouts in Hospital (top), Home (lower left), and Supermarket (lower right). Blue arrows show the guider route to the shared goal, and purple arrows show the fetcher route through pickup to that goal. Parallel execution uses both routes. Sequential execution uses the same locations but assigns both objectives to the fetcher.

before planner selection and copied into all four launch configurations. Planner comparisons match scene, seed, and schedule. Schedule comparisons match scene, seed, and planner source. All 90 sets contain the expected four records. Failed episodes remain in the success-rate denominator and are never replaced by additional runs.

5.3 Evaluation Metrics

Planner validity and operational success are reported separately, so a correct action schema is not mistaken for a completed mission. A plan is valid when it contains one parseable JSON object, the expected four actions, valid robot assignments, and a structure admissible under the selected schedule. Online runs must also retain the raw response and request metadata and show that neither fallback nor replay was used. Operational success follows Eq. (7) and requires goal arrival, pickup arrival, grasp, lift, hold, and delivery under the checks in Sec. 4. Path length and waypoint progress are trajectory measurements, not additional success criteria after goal arrival.

Makespan is compared only among matched runs completed by both schedules, which prevents early failure from appearing as a timing improvement. It is measured from the end of simulator stabilization until the last required terminal event. Relative time saving is $1 - T_{\text{par}}/T_{\text{seq}}$. Success rates are reported with Wilson 95% confidence intervals. Paired mean differences use a deterministic 20,000-sample percentile bootstrap interval and a 50,000-draw sign-flip test,

with scripted and online results analyzed separately. Trajectory logs also separate the fetcher’s unloaded visit, pickup approach, and loaded delivery from the guider route. Total distance sums the paths of all robots active in a condition and remains descriptive because two robots can travel farther in total while completing their routes concurrently.

6 Results

Across the 360 executions, planner source does not change the normalized action records, and mission outcomes are determined by scheduling and pre-transport manipulation.

6.1 Plan Validity and Operational Success

Plan validity does not imply mission success. All 180 online Qwen3.5 responses contain the raw output and prompt hash, parse as one JSON object, and pass the capability and schedule checks on the first request without fallback or replay. Each normalized plan matches the scripted plan for the same scene, seed, and schedule. The planner sources therefore provide the same controller inputs and produce identical task outcomes. For each source, sequential execution succeeds in 72/90 runs and parallel execution in 71/90 runs (Table 1). Planning latency is excluded from execution makespan because execution starts only after validation.

Table 1: Plan validity and operational success over 90 combinations of scene and seed per condition.

Planner	Schedule	Valid plans	Operational success	Wilson 95% interval [%]
Scripted	Sequential	90/90	72/90 (80.0%)	[70.6, 87.0]
Scripted	Parallel	90/90	71/90 (78.9%)	[69.4, 86.0]
Online Qwen3.5	Sequential	90/90	72/90 (80.0%)	[70.6, 87.0]
Online Qwen3.5	Parallel	90/90	71/90 (78.9%)	[69.4, 86.0]

6.2 Paired Makespan

Parallel execution is faster in every common-success case. The 56 matched cases comprise 13 Home, 22 Supermarket, and 21 Hospital missions. Mean makespan decreases from 164.78 to 96.79 s, giving a $1.70\times$ speedup and a 41.3% reduction. The paired mean reduction is 67.99 s, with a bootstrap 95% interval of [54.05, 82.04] s and a sign-flip test result of $p < 0.001$. Scripted and online results are identical because their normalized plans and task outcomes match. Table 2 shows the result by scene.

Table 2: Timing for cases completed by both schedules. Seq. denotes sequential scheduling, and Par. denotes parallel scheduling. The ratio is sequential makespan divided by parallel makespan.

Scene	Seq. success	Par. success	Pairs	Seq. [s]	Par. [s]	Ratio
Home	20/30	21/30	13	114.13	69.04	1.65×
Supermarket	26/30	25/30	22	84.02	67.94	1.24×
Hospital	26/30	25/30	21	280.74	144.20	1.95×

The source of the timing benefit depends on scene geometry. Temporal overlap is present in every scene shown in Figure 2. Figure 3 shows reductions of 39.5% in Home, 19.1% in Supermarket, and 48.6% in Hospital. In Home and Supermarket, the two robots travel farther in parallel than the fetcher travels alone, with totals of 24.95 versus 22.05 m and 16.76 versus 14.86 m. Their lower makespans therefore come from concurrent motion. In Hospital, assigning the unloaded visit to the guider also removes the fetcher’s long visit and return, reducing total travel from 68.25 to 40.60 m. The measured gain consequently reflects the implemented combination of role assignment and action overlap.

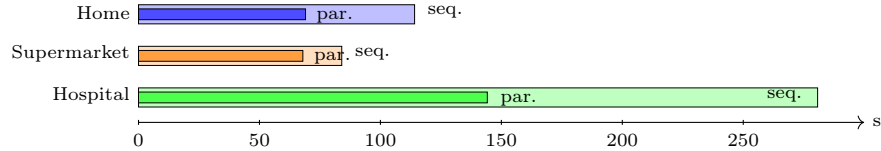


Fig. 3: Mean makespan for matched cases completed by both schedules. Light bars show sequential scheduling (Seq.), and dark inset bars show parallel scheduling (Par.).

6.3 Manipulation Outcomes

All operational failures arise during manipulation. Since the scripted and online planners produce identical physical outcomes, we count the 37 distinct scene, schedule, and seed failure configurations once. They fall into three grasp-retention modes. In 19 cases, the distance from the registered grasp reference to the end-effector frame origin exceeds 0.25 m during the hold, so the pre-transport gate is never reached. Another 15 retain final bilateral contact but exceed the 17° hold-tilt limit. The remaining three fail the final bilateral-contact check, including two that also exceed the tilt limit. These failures comprise 18 sequential and 19 parallel configurations. No episode fails plan validation or pickup arrival, and every guider in the parallel condition reaches its navigation goal. The difference between 72/90 and 71/90 operational successes therefore comes from fetcher manipulation. Figure 4 connects these state-based categories to their visible physical outcomes.

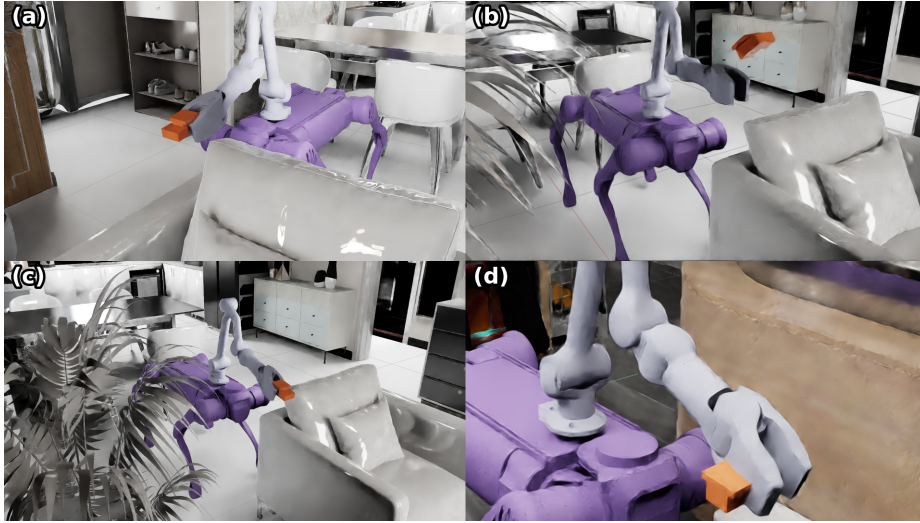


Fig. 4: Representative simulator captures of the manipulation outcomes: (a) verified retention, (b) object separation, (c) excessive hold tilt, and (d) final bilateral-contact loss.

Successful episodes verify grasp and lift before assisted transport. The 143 successful scripted episodes comprise 41 Home, 51 Supermarket, and 51 Hospital runs. All pass the bilateral-contact, lift, and hold checks before post-lift object synchronization. Mean peak and retained object rise are 69.89 and 62.05 mm in Home, 66.48 and 61.89 mm in Supermarket, and 69.43 and 63.05 mm in Hospital. Corresponding online runs produce the same measurements under the normalized plans, so Table 3 reports one planner source.

Table 3: Object rise in successful scripted episodes, measured before post-lift object synchronization.

Scene	Successful episodes	Peak rise [mm]	Retained rise [mm]
Home	41	69.89	62.05
Supermarket	51	66.48	61.89
Hospital	51	69.43	63.05

7 Discussion

The language interface is consistent within the tested schema. All 180 online responses match their scripted counterparts without fallback. This evidence is limited to the tested templates and fixed action inventory.

The timing gain has two sources. In Home and Supermarket, the parallel team travels farther in total yet finishes sooner, showing the effect of temporal overlap. In Hospital, role reassignment also removes the fetcher’s long unloaded visit and return. The 41.3% reduction therefore reflects the implemented role assignment, action overlap, and scene geometry, not faster controllers.

Manipulation remains the bottleneck in the guide-and-fetch pipeline. Operational success includes all 90 cases per condition, while paired makespan uses only the 56 common successes. Every failure occurs during pre-transport grasp, lift, or retention. None originates in plan validation or pickup arrival, and every parallel guider reaches its goal. Stable object handling therefore limits completion of the combined mission.

8 Limitations and Future Work

Language generalization. Evidence for the language interface is limited to four paraphrase templates per schedule and a fixed action inventory. The evaluation does not cover unseen schemas, infeasible or conflicting requests, or plan repair. Future experiments should introduce held-out instructions, larger action schemas, deliberately invalid assignments, and state feedback that requires re-planning.

Manipulation scope. The manipulation evidence covers one lightweight handled object under bounded pose, mass, and friction perturbations. Physical checks end before assisted transport begins. Future work should test varied objects, require unaided retention throughout loaded motion, and verify delivery until object release at the destination.

Deployment realism and human guidance. The deployment study is limited to three registered scenes, controlled routes, disabled background physics, and kinematic root transport. Destination arrival is only a proxy for guidance because human following and escort safety are not modeled. Future work should add dynamic base control, changing obstacles, and unseen layouts before user studies examine task time, uncertainty, workload, and perceived risk with blind and low-vision participants.

9 Conclusion

This work introduced GUIDEFETCH, a framework for coordinating heterogeneous guider and fetcher robots from a single natural-language request. The framework instantiates a schedule-conditioned action schema, checks each assignment against robot capabilities, and coordinates sequential or parallel execution through state-based events. It keeps language-specified intent, robot execution, and verified completion distinct. In the motivating scenario, one robot visits the destination while another performs state-verified object acquisition and assisted delivery. GUIDEFETCH thus integrates role specialization and concurrent action without treating a valid plan as evidence of mission success.

10 Acknowledgment

This work was supported in part by the Ministry of Science, Research and the Arts of Baden-Württemberg (MWK) through the Cooperative Graduate School Accessibility through AI-based Assistive Technology (KATE) under Grant BW6-03, in part by funding from the pilot program Core-Informatics of the Helmholtz Association (HGF), in part by Karlsruhe House of Young Scientists (KHYS), and in part by the Helmholtz Association Initiative and Networking Fund on the HAICORE@KIT and HOREKA@KIT partition. This work is supported in part by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - SFB 1574 - 471687386. This work was also supported in part by the National Natural Science Foundation of China (Grant No. 62473139), in part by the Hunan Provincial Research and Development Project (Grant No. 2025QK3019), and in part by the State Key Laboratory of Autonomous Intelligent Unmanned Systems (the opening project number ZZKF2025-2-10).

References

1. Assistance Dogs International: ADI terms and definitions. <https://assistancedogsinternational.org/resources/adi-terms-definitions/> (2026), accessed: 2026-08-05
2. Bi, Z., Zhang, Y., Chen, K., Zhao, G., Li, Y., Ma, J.: ALORE: Autonomous large-object rearrangement with a legged manipulator. arXiv preprint arXiv:2602.04214 (2026)
3. Chang, M., Chhablani, G., Clegg, A., et al.: PARTNR: A benchmark for planning and reasoning in embodied multi-agent tasks. In: International Conference on Learning Representations (2025)
4. Chen, J., Yu, C., Zhou, X., Xu, T., Mu, Y., Hu, M., Shao, W., Wang, Y., Li, G., Shao, L.: EMOS: Embodiment-aware heterogeneous multi-robot operating system with LLM agents. In: International Conference on Learning Representations (2025)
5. Chen, W., Liu, S.C., Li, Q., Li, Y.H., Zhang, J.: Enhancing task-oriented robotic grasping via 3D affordance grounding from vision-language models. *Complex & Intelligent Systems* **12**(1), 42 (2026). <https://doi.org/10.1007/s40747-025-02169-0>
6. Chen, Y., Arkin, J., Zhang, Y., Roy, N., Fan, C.: Scalable multi-robot collaboration with large language models: Centralized or decentralized systems? In: IEEE International Conference on Robotics and Automation. pp. 4311–4317 (2024). <https://doi.org/10.1109/ICRA57147.2024.10610676>
7. Guo, P., Mai, Z., Xu, Z., Zhang, K., Luu, Q.K., Zhang, H., Miao, Z., Ajoudani, A., Kingston, Z., Qiu, Q., et al.: PPlanAR: Planning-language-grounded agentic reasoning for robot manipulation. In: The 2nd Workshop on Foundation Models Meet Embodied Agents at CVPR 2026 (2026)
8. Huang, C., Mees, O., Zeng, A., Burgard, W.: Multimodal spatial language maps for robot navigation and manipulation. *The International Journal of Robotics Research* p. 02783649251351658 (2025). <https://doi.org/10.1177/02783649251351658>
9. Huang, W., Xia, F., Xiao, T., et al.: Inner monologue: Embodied reasoning through planning with language models. In: Conference on Robot Learning. pp. 1769–1782 (2023)

10. Ichter, B., Brohan, A., Chebotar, Y., et al.: Do as I can, not as I say: Grounding language in robotic affordances. In: Proceedings of the 6th Conference on Robot Learning. pp. 287–318 (2023)
11. Kang, L., Song, X., Zhou, H., Qin, Y., Yang, J., Liu, X., Torr, P., Bai, L., Yin, Z.: VIKI-R: Coordinating embodied multi-agent cooperation via reinforcement learning. *Advances in Neural Information Processing Systems* **38** (2025)
12. Kannan, S.S., Venkatesh, V.L.N., Min, B.C.: SMART-LLM: Smart multi-agent robot task planning using large language models. *arXiv preprint arXiv:2309.10062* (2024)
13. Kim, J.T., Yu, W., Kothari, Y., Walker, B., Tan, J., Turk, G., Ha, S.: Transforming a quadruped into a guide robot for the visually impaired: Formalizing wayfinding, interaction modeling, and safety mechanism. In: Proceedings of the 7th Conference on Robot Learning, Proceedings of Machine Learning Research, vol. 229, pp. 2288–2303. PMLR (2023), <https://proceedings.mlr.press/v229/kim23c.html>
14. King, C.H., Chen, T.L., Fan, Z., Glass, J.D., Kemp, C.C.: Dusty: An assistive mobile manipulator that retrieves dropped objects for people with motor impairments. *Disability and Rehabilitation: Assistive Technology* **7**(2), 168–179 (2012). <https://doi.org/10.3109/17483107.2011.615374>
15. Li, Y., Qin, K., Chen, W., Wu, H.: HOSG-Nav: Hierarchical open-vocabulary semantic graph navigation for language-guided global planning in 3D gaussian scenes. *Electronics* **15**(10), 2179 (2026). <https://doi.org/10.3390/electronics15102179>
16. Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as policies: Language model programs for embodied control. In: IEEE International Conference on Robotics and Automation. pp. 9493–9500 (2023)
17. Lima, O., Vinci, M., Günther, M., Renz, M., Sung, A., Stock, S., Brust, J., Niecksch, L., Yi, Z., Igelbrink, F., et al.: Agentic AI for robot control: Flexible but still fragile. In: Proceedings of the AAAI Symposium Series. vol. 8, pp. 465–473 (2026)
18. Liu, K., Tang, Z., Wang, D., Wang, Z., Li, X., Zhao, B.: COHERENT: Collaboration of heterogeneous multi-robot system with large language models. In: IEEE International Conference on Robotics and Automation (2025)
19. Mandi, Z., Jain, S., Song, S.: RoCo: Dialectic multi-robot collaboration with large language models. In: IEEE International Conference on Robotics and Automation. pp. 286–299 (2024). <https://doi.org/10.1109/ICRA57147.2024.10610855>
20. Puig, X., Undersander, E., Szot, A., et al.: Habitat 3.0: A co-habitat for humans, avatars and robots. In: International Conference on Learning Representations (2024)
21. Qayyum, M.A., Doore, S.A.: Toward collaborative navigation: A voice-based interface for quadruped guide robots. In: Companion Proceedings of the 21st ACM/IEEE International Conference on Human-Robot Interaction. pp. 964–968 (2026). <https://doi.org/10.1145/3776734.3794537>
22. Qi, Z., Zhang, W., Ding, Y., Dong, R., Yu, X., Li, J., Xu, L., Li, B., He, X., Fan, G., et al.: SoFar: Language-grounded orientation bridges spatial reasoning and object manipulation. *Advances in Neural Information Processing Systems* **38**, 76367–76412 (2025)
23. Qin, Y., Kang, L., Song, X., Yin, Z., Liu, X., Liu, X., Zhang, R., Bai, L.: Robo-Factory: Exploring embodied agent collaboration with compositional constraints. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 10075–10085 (2025)

24. Qwen Team: Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5> (February 2026), accessed: 2026-08-05
25. Rana, K., Haviland, J., Garg, S., Abou-Chakra, J., Reid, I., Sünderhauf, N.: Say-Plan: Grounding large language models using 3D scene graphs for scalable robot task planning. In: Proceedings of the 7th Conference on Robot Learning. pp. 23–72 (2023)
26. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
27. Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., Garg, A.: ProgPrompt: Generating situated robot task plans using large language models. In: IEEE International Conference on Robotics and Automation. pp. 11523–11530 (2023). <https://doi.org/10.1109/ICRA48891.2023.10161317>
28. Tripathi, M.M., Huang Xin, Li, Q.: Repair2Skill: A vision–language–action framework for robotic furniture repair. In: Proceedings of the AAAI Symposium Series. vol. 8, pp. 109–117 (2026). <https://doi.org/10.1609/aaais.v8i1.42525>
29. Wang, H., Chen, J., Huang, W., et al.: GRUtopia: Dream general robots in a city at scale. arXiv preprint arXiv:2407.10943 (2024)
30. Wang, X., Yang, X., Xu, Y., Wu, Y., Li, Z., Zhao, N.: AffordBot: 3D fine-grained embodied reasoning via multimodal large language models. Advances in Neural Information Processing Systems **38**, 140367–140387 (2025)
31. Xie, J., Zhang, W., Chen, H., Zeng, J., Gao, Y., Xu, Z., Hashimoto, K.: Centralized LLM-driven multi-robot coordination for cooperative object transportation. In: 2025 IEEE International Conference on Advanced Robotics and its Social Impacts (ARSO). pp. 368–373. IEEE (2025)
32. Xue, D., Zhou, X., Wang, M., Liu, F.: Formation control and path planning of multi-robot systems via large language models. Science China Information Sciences **68**(5), 150205 (2025). <https://doi.org/10.1007/s11432-024-4290-4>
33. Yang, T., Zhong, R., Shi, W.: LaViCoN: Language and vision cooperative navigation for assistive mobile robots. In: 2026 IEEE 4th International Conference on Mobility, Operations, Services and Technologies (MOST). pp. 57–66. IEEE (2026). <https://doi.org/10.1109/MOST69733.2026.00016>
34. Zu, L., Lin, L., Fu, S., Zhao, N., Zhou, P.: Collaborative tree search for enhancing embodied multi-agent collaboration. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 29513–29522 (2025). <https://doi.org/10.1109/CVPR52734.2025.02748>